



Parte 2 - 12.5 valores

Pretende-se modelar um sistema de automação doméstica, representado pela classe `SmartHome`. O sistema gere um conjunto de divisões da casa, um conjunto de dispositivos inteligentes instalados nessas divisões e um registo de agendamentos que controlam automaticamente os dispositivos.

As divisões, instâncias de `Divisao`, possuem um identificador sequencial atribuído automaticamente e um nome (ex.: "Sala", "Cozinha", "Quarto"). Cada divisão contém um conjunto de dispositivos.

Cada dispositivo possui um identificador sequencial único (global, não por divisão), um nome e um estado (ligado/desligado). Existem três tipos concretos de dispositivos: os `Iluminacao`, que guardam o nível de brilho (inteiro de 0 a 100) e a temperatura de cor em Kelvin, e cujo consumo actual é proporcional ao brilho (ou zero se desligada); os `Termostato`, que guardam a temperatura alvo e a temperatura actual (`double`), com um consumo fixo quando ligado e as temperaturas diferem; e as `Tomada`, que guardam a potência máxima (`double`) e cujo consumo actual é a potência máxima se ligada, ou zero caso contrário.

Os agendamentos, instâncias de `Agendamento`, têm um identificador sequencial atribuído automaticamente e definem: o dispositivo a controlar, a hora de disparo (`LocalTime`), a acção a efectuar (ligar ou desligar, representada por um `boolean`), e os dias da semana em que o agendamento está activo (`Set<DayOfWeek>`). Um `LocalTime` é um objecto imutável que representa tempo, e guarda horas, minutos e segundos. `DayOfWeek` é um enum com os valores Monday, Tuesday, Wednesday, Thursday, Friday, Saturday e Sunday. Considere os seguintes excertos de código:

```
public class SmartHome implements Serializable {
    private Map<Integer, Divisao> divisoes; // idDivisao -> Divisao
    // outras variáveis de instância...
}

public class Agendamento implements Comparable<Agendamento>, Serializable {

    // outras variáveis...

    public boolean deveDisparar(LocalTime hora, DayOfWeek dia) {...}
}

public abstract class Dispositivo implements Serializable {
    private int id;
    private String nome;
    private boolean ligado;

    ...
    public abstract double consumoAtual(); // consumo em Watts
}
```

Considere que a estratégia de associação entre `SmartHome` e as suas divisões e agendamentos é de **composição**.

Assuma, para as perguntas seguintes, que os métodos usuais (`get`, `set`, `equals`, `clone`, `hashCode`, `toString`, ...) estão disponíveis nas classes onde sejam necessários, a menos que sejam solicitados, e responda às questões:



Questão 6

- 1) Efectue a declaração das variáveis de instância das classes `SmartHome`, `Divisao` e `Agendamento` e **justifique brevemente** a escolha das estruturas de dados que faz em `SmartHome` e das relações de associação entre as classes.
- 2) Codifique o método `clone()` da classe `Agendamento`, tendo em conta que a associação entre `Agendamento` e `Dispositivo` é de **composição**.

☐ 0 ☐ .2 ☐ .4 ☐ .5 ☐ .6 ☐ .8 ☐ 1 *Reservado aos docentes*



Continuação

(espaço adicional)

☐0 ☐.2 ☐.4 ☐.5 ☐.6 ☐.8 ☐1 *Reservado aos docentes*



Questão 7

a) Codifique o método `public void adicionaDispositivo(int idDivisao, Dispositivo d) throws...`, da classe `SmartHome`, que instala um dispositivo numa divisão. O mesmo dispositivo (pelo id) não pode estar presente em mais do que uma divisão. Identifique todas as situações de inconsistência a verificar.

b) Codifique o método `public void atualizaAgendamento(int idAgendamento, LocalTime novaHora, Set<DayOfWeek> diasSemana) throws...`, da classe `SmartHome`, que actualiza a hora e os dias de um agendamento existente. A nova hora não pode ser nula, o conjunto de dias não pode ser nulo nem vazio, e o agendamento deve existir. Identifique todas as situações de inconsistência a verificar, tendo em conta a estratégia de composição.

☐ 0 ☐ .2 ☐ .4 ☐ .5 ☐ .6 ☐ .8 ☐ 1 *Reservado aos docentes*



Continuação

(espaço adicional)

☐

0

☐

.2

☐

.4

☐

.5

☐

.6

☐

.8

☐

1

Reservado aos docentes



Questão 8

Codifique o método `public List<Divisao> rankingDivisoesPorConsumo()`, da classe `SmartHome`, que devolve a lista de divisões ordenada de forma decrescente pelo consumo total actual (soma de `consumoAtual()` de todos os dispositivos *ligados* na divisão). Em caso de empate no consumo, as divisões surgem por ordem alfabética do nome. Divisões sem dispositivos ligados devem aparecer no final, igualmente ordenadas pelo nome.

☐0 ☐.2 ☐.4 ☐.5 ☐.6 ☐.8 ☐1 Reservado aos docentes



Questão 9

Pretende-se que alguns dispositivos da plataforma possam ser monitorizados, registando automaticamente um histórico de eventos (ex.: hora e descrição de cada ligação/desligação). Para o efeito, considera-se a seguinte classe auxiliar já disponível:

```
public class MonitorizavelSupport implements Serializable {  
    public void registaEvento(LocalDateTime momento, String descricao) {...}  
    public List<String> historicoEventos() {...}  
    public String ultimoEvento() {...}  
    public int totalEventos() {...}  
}
```

1) Defina `Monitorizavel`, que declara os métodos `registarEvento`, `historicoEventos`, `ultimoEvento` e `totalEventos`. Redefina a classe `Iluminacao` para que seja compatível com este tipo, identificando as variáveis de instância adicionais e codificando os quatro métodos com recurso a `MonitorizavelSupport`. Justifique por que razão esta abordagem é preferível a uma classe abstracta neste contexto.

2) Codifique o método `public List<Monitorizavel> dispositivosMonitorizados()`, da classe `SmartHome`, que devolve todos os dispositivos da plataforma que são do tipo `Monitorizavel`, ordenados de forma decrescente pelo número total de eventos registados.

☐ 0 ☐ .2 ☐ .4 ☐ .5 ☐ .6 ☐ .8 ☐ 1 Reservado aos docentes



Continuação

(espaço adicional)

☐0 ☐.2 ☐.4 ☐.5 ☐.6 ☐.8 ☐1 *Reservado aos docentes*